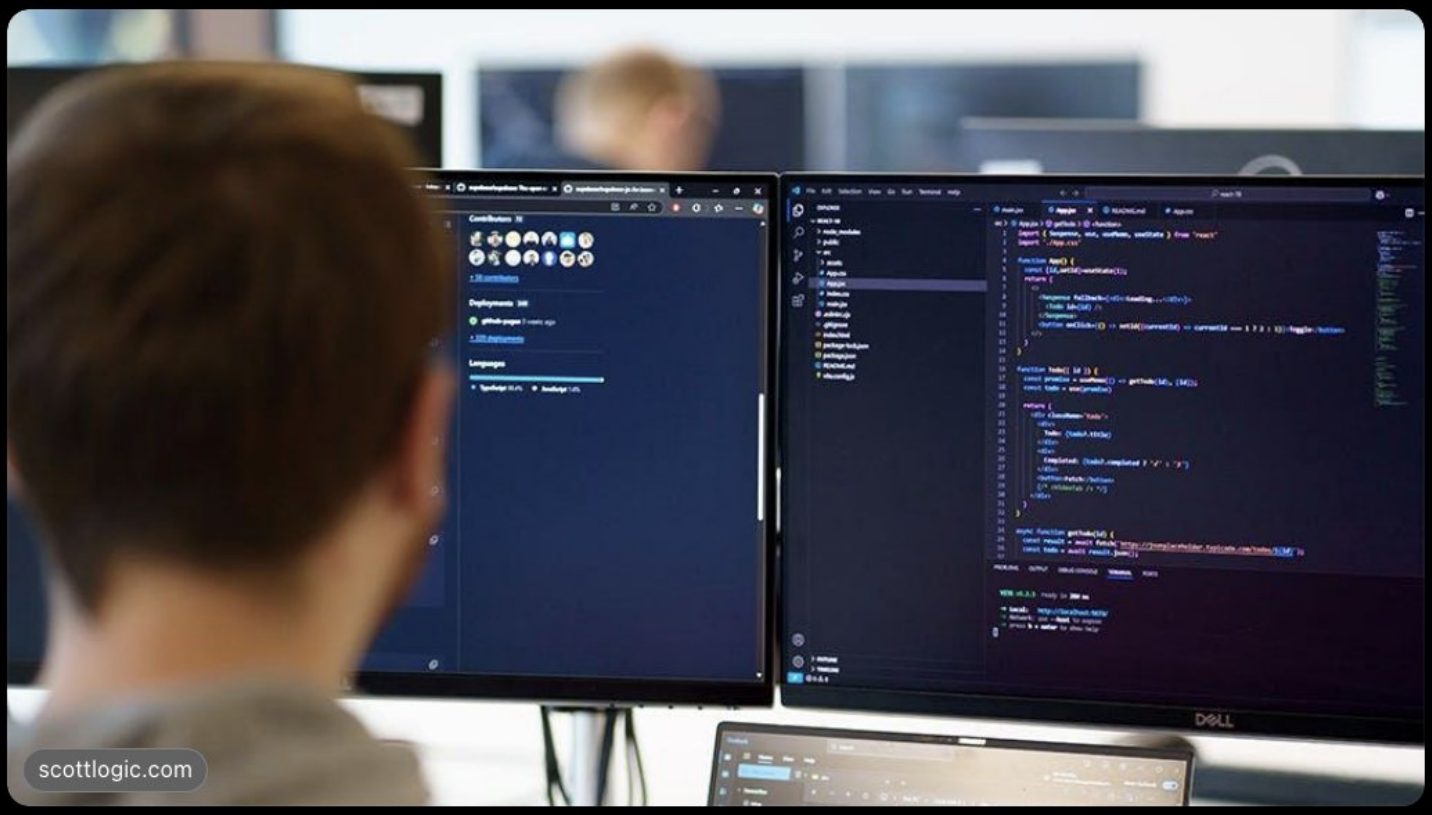
1. Complete CI/CD architecture on GCP



Stage 1: Developer pushes new code to GitHub



What happens?

The developer changes the checkout API code, fixes the bug, and pushes the changes to GitHub.

For example:

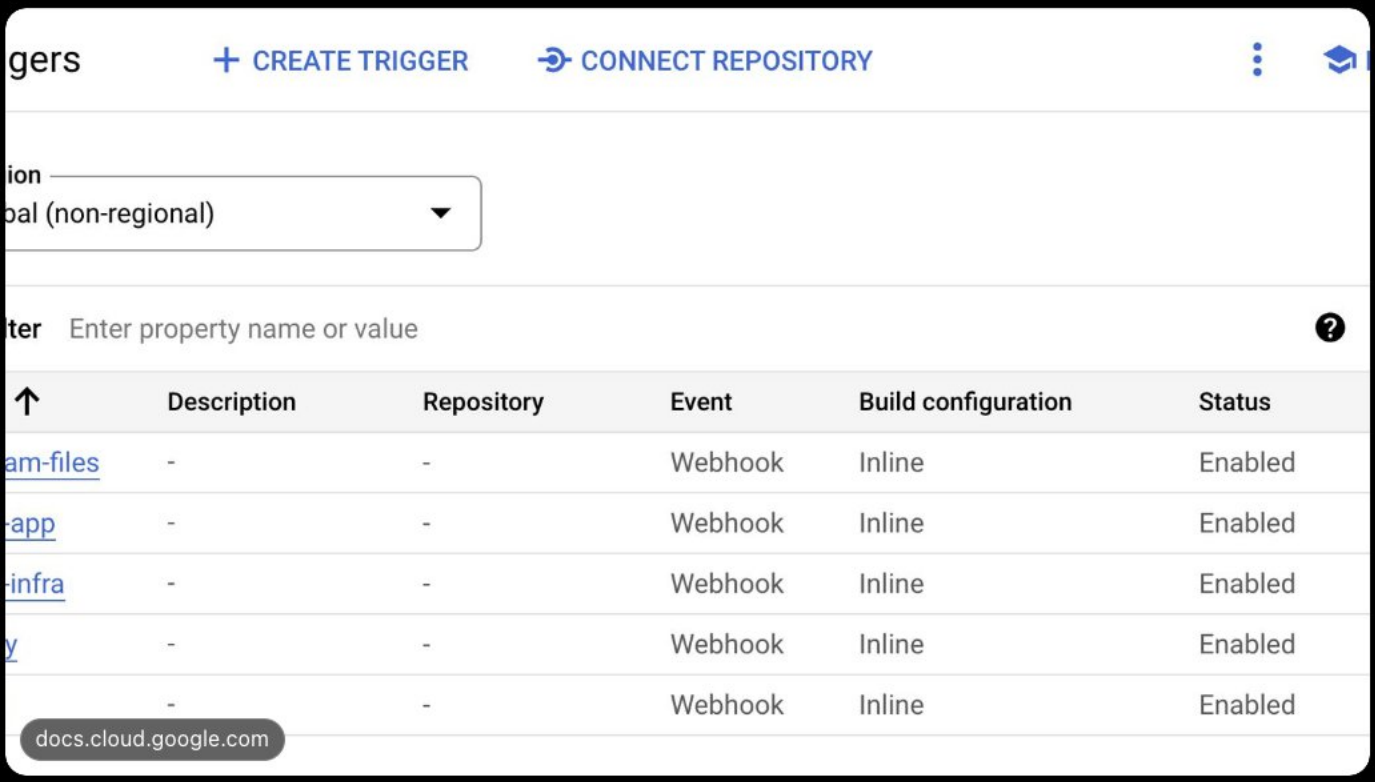
```bash
git add .
git commit -m "Fixed checkout API issue"
git push origin main
```

GitHub now contains the updated source code.

In simple words: The developer uploads the latest code to the shared repository so the CI/CD process can begin.

Tools: Git, GitHub.

Stage 2: GitHub triggers Google Cloud Build



What happens?

Cloud Build needs to know when new code arrives.

You configure a Cloud Build trigger connected to the GitHub repository and select the appropriate branch or pull-request event.

When the developer pushes a change to the configured branch, the trigger starts a build. Cloud Build runs the pipeline steps defined in a `cloudbuild.yaml` file or configured through the build setup.

In simple words: GitHub tells Google Cloud Build, "New code has arrived. Start the build and checks."

Tools: GitHub, Cloud Build triggers, `cloudbuild.yaml`.

Cloud Build runs the pipeline tasks; it doesn't automatically mean that the application has been deployed yet.

Stage 3: Cloud Build checks out and builds the application

What happens?

Cloud Build retrieves the source code and runs the commands needed to build the application.

For example, for a Java application:

```bash
mvn clean package
```

This command compiles the Java code, downloads dependencies if needed, runs the configured build lifecycle, and packages the application.

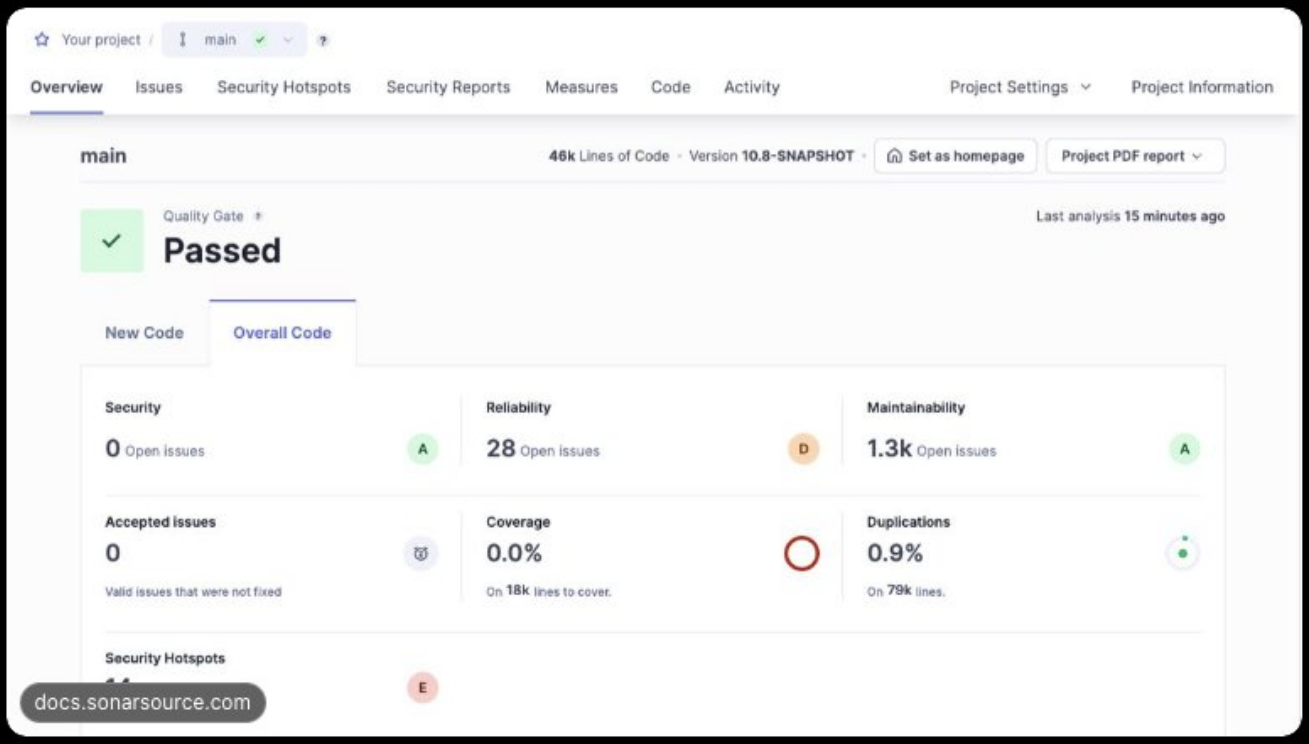
For a Python application, the steps might instead install dependencies and prepare the application package.

In simple words: Google Cloud Build takes the latest code and checks whether it can be built successfully.

If the build fails, the pipeline normally stops, and the team investigates the error.

Tools: Cloud Build, Maven or Gradle for Java, pip or Poetry for Python.

Stage 4: Automated tests, code-quality checks and security scans



What happens?

After the build, the pipeline runs checks to find problems before release.

Automated tests

Tools: JUnit, pytest, Postman/Newman.

These check whether the checkout API and other application functions work correctly.

Code-quality checks

Tool: SonarQube.

Checks for code-quality problems, bugs, and certain security issues.

Security scanning

Tools: Trivy, Snyk, or other configured scanners.

Checks for known vulnerabilities in dependencies, source code, and container images, depending on the scanner and configuration.

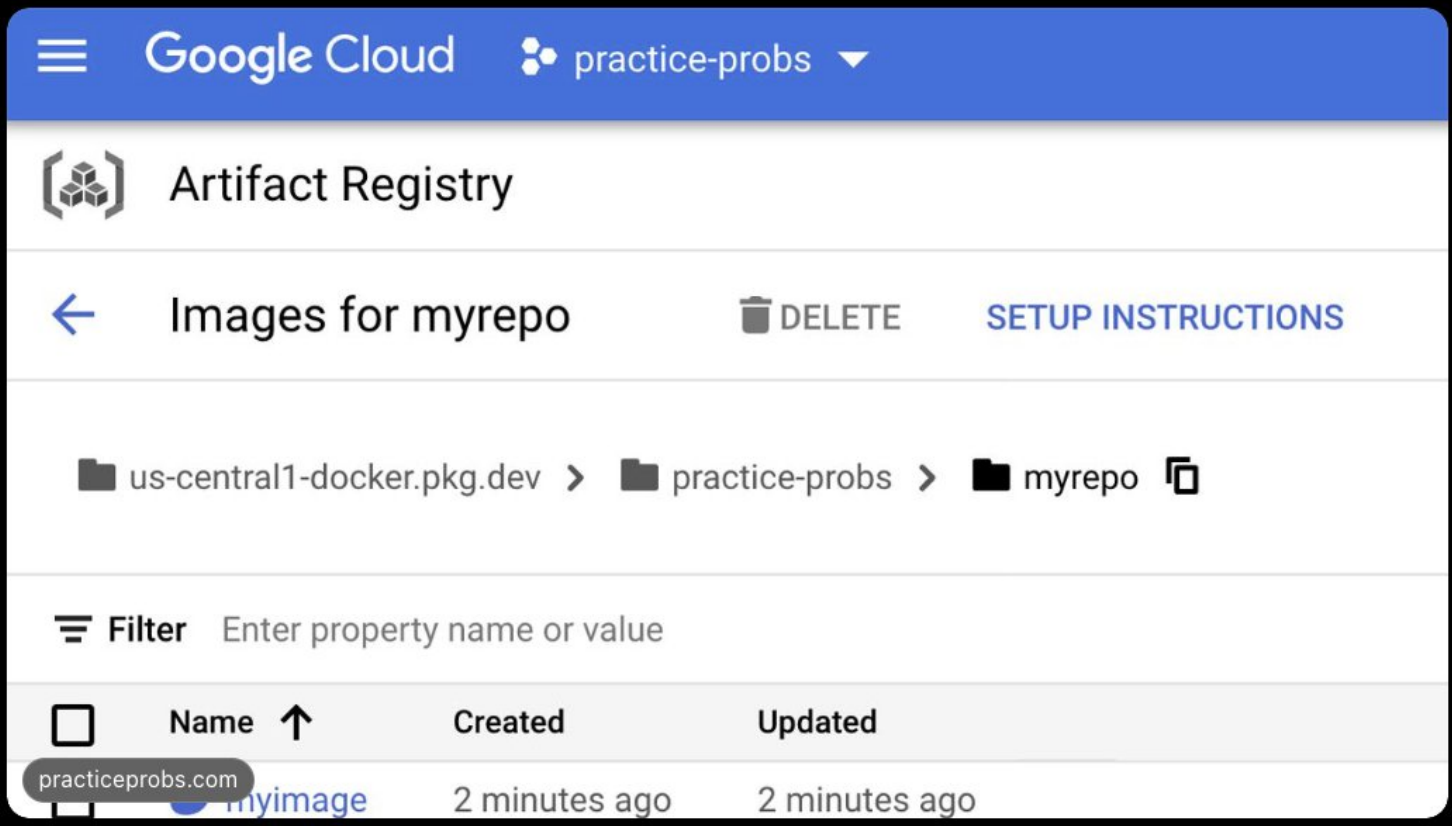
In simple words: We check that the application works, the code meets quality requirements, and known security problems are not being introduced.

If a required test or scan fails, the pipeline should stop before releasing the image.

Tools: Cloud Build, JUnit/pytest, SonarQube, Trivy.

Note that SonarQube and Trivy are additional tools you configure; they are not automatically enabled simply because you use Cloud Build.

Stage 5: Cloud Build creates a Docker image and pushes it to Artifact Registry



This stage has two parts.

Part A: Build the Docker image

After the required checks pass, Cloud Build uses the application's `Dockerfile` to create a container image.

For example:

```bash
docker build -t checkout-app:v12 .
```

Here, `v12` is an example version tag.

The image packages the application and the runtime components needed to run it.

Part B: Push the image to Artifact Registry

Artifact Registry is Google Cloud's service for storing container images and other software artifacts.

Cloud Build authenticates using its configured service account and pushes the image to the appropriate repository.

For example:

```bash
docker push us-central1-docker.pkg.dev/PROJECT_ID/apps/checkout-app:v12
```

This is an illustrative image path. `PROJECT_ID`, the repository, region, and image name must match your configuration.

In simple words: Cloud Build packages the application into a Docker image and stores it in Artifact Registry. The deployment system can then use that exact image.

Remember:

- Cloud Build builds and tests the application.
- Artifact Registry stores the image.
- GKE or Cloud Run runs the application.

Stage 6: Deploy the application to GKE or Cloud Run

Google Cloud supports different deployment options. The right one depends on how the application is designed.



Option A: Google Kubernetes Engine (GKE)

GKE is suitable when the application needs Kubernetes features such as Deployments, Pods, Services, custom networking, and more control over orchestration.

The deployment system updates the Kubernetes Deployment to use the new image from Artifact Registry.

Tools: GKE, Kubernetes, `kubectl`, Helm, Cloud Deploy or Cloud Build.



Option B: Cloud Run

Cloud Run is suitable for many containerized applications when you want Google Cloud to manage the underlying serving infrastructure.

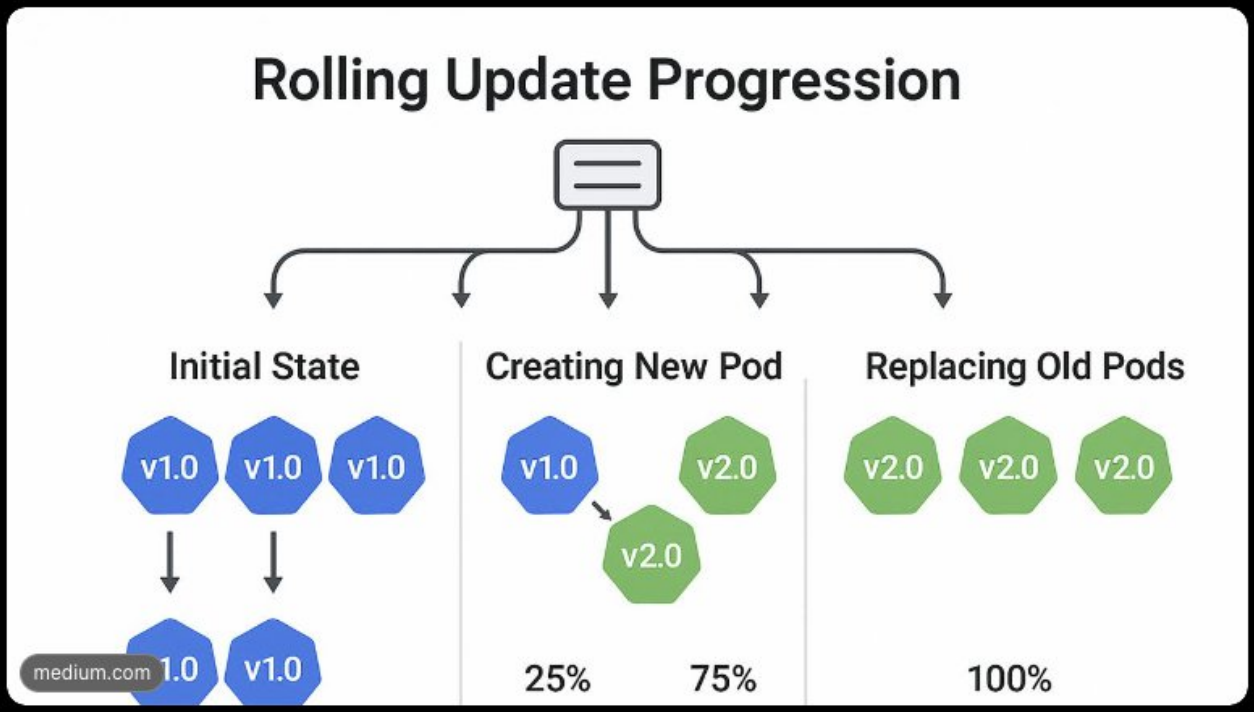
The pipeline deploys the new container image as a new revision. Traffic can be directed to that revision after validation.

Tools: Cloud Run, Artifact Registry, Cloud Build or Cloud Deploy.

In simple words: The deployment system tells Google Cloud where and how to run the new application version.

For the rest of the explanation, we'll continue with **GKE**.

Stage 7: GKE performs a rolling update



What happens?

Imagine your checkout application is running in three Kubernetes Pods using image `v11`.

The new image `v12` is now stored in Artifact Registry.

The deployment updates the GKE Deployment to use `v12`. Kubernetes gradually creates new Pods and replaces old Pods according to the rollout settings.

Before deployment

Pod 1	Pod 2	Pod 3
v11	v11	v11

↓↑ Rolling update

After deployment

Pod 1	Pod 2	Pod 3
v12	v12	v12

Illustration: old Pods are replaced with new Pods gradually. Actual rollout behavior depends on capacity, readiness, and Deployment settings.

In simple words: Kubernetes replaces the old application instances with the new ones gradually, helping the application continue serving traffic during deployment.

A rolling update does not automatically guarantee zero downtime. The application needs appropriate readiness probes, enough capacity, and correct rollout settings.

Stage 8: Validate the new deployment

What happens?

Once GKE starts the new version, the pipeline checks whether the application is healthy and the important functionality works.

There are two main checks:

- **Health checks:** Kubernetes readiness probes determine whether a Pod is ready to receive traffic.
- **Smoke tests:** Automated tests send basic requests to the deployed application and verify important behavior, such as the checkout API returning a valid response.

For example:

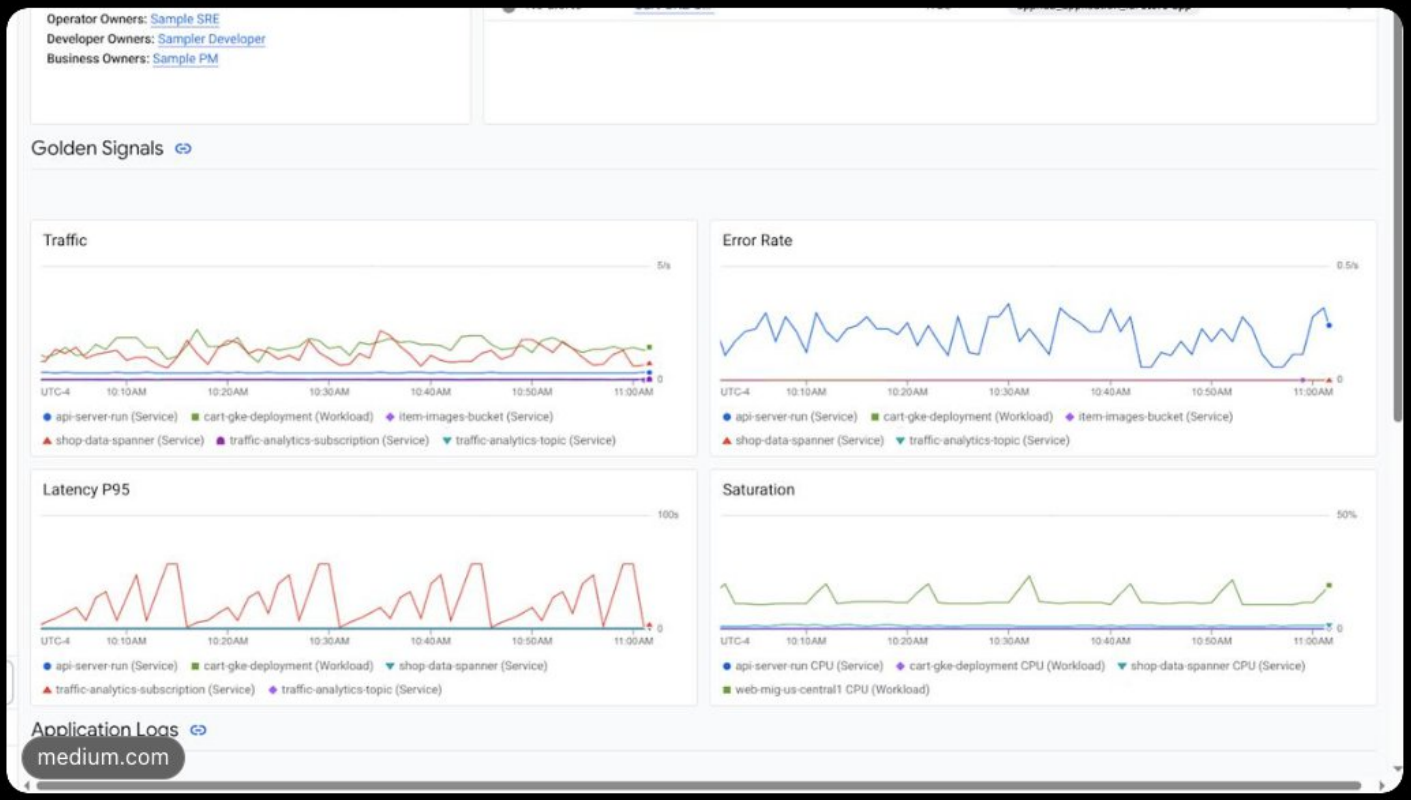
```bash
curl -f https://example.com/health
```

This URL is only an example; your application must implement the endpoint, and the pipeline must use the real URL.

In simple words: We check whether the application is actually working after deployment, rather than assuming it is healthy just because the deployment command succeeded.

Tools: Kubernetes probes, curl, Postman/Newman, automated test scripts, Cloud Deploy verification where configured.

Stage 9: Monitor the application using Google Cloud monitoring tools



What happens?

After deployment, the team monitors how the application behaves under real traffic.

Google Cloud provides tools for metrics, logs, alerts, and troubleshooting.

Tool	What it does
Cloud Monitoring	Tracks metrics, dashboards, uptime checks, and alerting.
Cloud Logging	Collects and searches application and infrastructure logs.
Cloud Trace	Helps investigate request latency and distributed tracing.
Managed Service for Prometheus	Collects Prometheus-compatible metrics, useful for Kubernetes workloads.
Grafana	Can visualize metrics through configured data sources.

For example, after deploying `v12`, the team might notice an increase in checkout API errors or response time.

They use the dashboards, alerts, and logs to investigate the problem.

In simple words: Monitoring helps the DevOps team identify issues that occur after the new version is deployed.

Stage 10: Roll back if the new release causes errors

What happens?

Suppose the new checkout version `v12` causes errors in production.

The team investigates the logs and metrics. If the new release is responsible, the team restores the previous stable version, `v11`, while fixing the problem.

For a compatible Kubernetes Deployment on GKE, one rollback command is:

```bash
kubectl rollout undo deployment/checkout-api
```

This rolls back to the previous recorded Deployment revision, provided it is available and the deployment is managed in a compatible way.

In a Cloud Deploy setup, the team can use the configured rollout and release-management workflow. In a Cloud Run setup, the team can route traffic back to a previous healthy revision.

In simple words: If the new version breaks the application, we restore the previous working version to recover service while investigating the root cause.

Rollback does not automatically undo database changes or external side effects, so those need a separate recovery plan.

3. Where does Terraform fit into the GCP pipeline?

Terraform is used to create and manage Google Cloud infrastructure through code.

For example, Terraform can provision:

- VPC networks, subnets, and firewall rules.
- GKE clusters and node pools.
- Artifact Registry repositories.
- IAM service accounts and permissions.
- Cloud Run services and other supported resources.

In simple words: Terraform prepares the infrastructure where the application will run. Cloud Build and the deployment system then build and release the application.

Terraform is often managed in a separate infrastructure pipeline, rather than being run every time a developer changes application code.

5. Which GCP tools should you remember for an interview?

Pipeline stage	GCP tool	Main purpose
Source control	GitHub / Cloud Source Repositories where available	Store and manage code
CI trigger	Cloud Build	Start builds on configured events
Build and tests	Cloud Build	Execute build and test steps
Code quality	SonarQube	Analyze code quality
Container scanning	Trivy or another scanner	Find known vulnerabilities
Image storage	Artifact Registry	Store container images
Infrastructure	Terraform	Provision GCP resources
Kubernetes deployment	GKE	Run and orchestrate containers
Release management	Cloud Deploy	Manage delivery and rollout to supported targets
Serverless container deployment	Cloud Run	Run containers without managing a Kubernetes cluster
Metrics and alerts	Cloud Monitoring	Monitor application and infrastructure health
Logs	Cloud Logging	Search and investigate logs

One important distinction: **Cloud Build and Cloud Deploy are not the same tool.** Cloud Build executes build and test steps; Cloud Deploy manages delivery and rollout workflows to supported targets, including GKE and Cloud Run.

6. How to explain the GCP CI/CD pipeline in an interview

Writing

“In my project, when a developer pushed new code to GitHub, a Cloud Build trigger started the CI pipeline. Cloud Build checked out the source code, built the application, and ran the automated tests.

After the required checks passed, we performed code-quality and security scans. Then we built a Docker image and pushed it to Artifact Registry.

For deployment, we used GKE to run the containerized application. The deployment process updated the Kubernetes Deployment with the new image, and Kubernetes gradually replaced the old Pods with the new ones.

After deployment, we verified the application using health checks and smoke tests. We used Cloud Monitoring and Cloud Logging to monitor performance and investigate issues. If a release caused a serious problem, we could roll back to the previous stable version using our configured deployment process.

We used Terraform to manage the infrastructure. The main goal was to automate the build, testing, deployment, and verification process and reduce manual work and deployment risk.”

3. GCP example — where Terraform sits

Now let's use the same e-commerce application on Google Cloud.

Here, the tools are Cloud Build, Artifact Registry, GKE, and Terraform.



What exactly does Terraform create on GCP?

GCP resource	Why we need it
VPC and subnets	Provide network connectivity for application resources.
IAM service accounts and permissions	Control access between services and users.
GKE cluster and node pools	Run the containerized application on Kubernetes.
Artifact Registry repository	Store Docker images.
Load balancer and networking resources	Route traffic to the application.
Cloud SQL, if required	Store application data.

The process is similar to AWS. Terraform creates the environment, and the application pipeline deploys new application versions into that environment.

What happens in the GCP Terraform pipeline?

1. You write Terraform files describing the GCP resources.
2. You push the files to GitHub.
3. A Cloud Build trigger starts the infrastructure pipeline.
4. Cloud Build initializes Terraform and validates the configuration.
5. Terraform generates a plan showing which resources will be created, changed, or destroyed.
6. The team reviews and approves the plan when required.
7. Terraform applies the approved changes using an appropriately authorized service account.
8. GKE and the other required resources are ready for application deployment.

Example commands:

```bash
terraform init
terraform validate
terraform plan
terraform apply
```

For production, configure Terraform state in a secure remote backend, such as a Google Cloud Storage bucket, and protect state access. Use least-privilege IAM permissions and approval controls for infrastructure changes.

4. AWS vs GCP — Terraform and CI/CD tools

Purpose	AWS	GCP
Infrastructure as Code	Terraform	Terraform
CI pipeline	Jenkins or AWS-integrated CI tooling	Cloud Build or Jenkins
Container image creation	Docker	Docker
Image registry	Amazon ECR	Artifact Registry
Kubernetes platform	Amazon EKS	Google Kubernetes Engine (GKE)
Deployment management	Jenkins, Helm, Argo CD or AWS deployment services	Cloud Deploy, Cloud Build, Helm or Argo CD
Monitoring	CloudWatch, Prometheus, Grafana	Cloud Monitoring, Cloud Logging, Managed Service for Prometheus, Grafana
Infrastructure state	S3 remote backend, configured for Terraform state	GCS remote backend

5. The most important concept to understand

There are **two different pipelines** in many real-world projects.

Pipeline 1

Infrastructure pipeline

Triggered when infrastructure code changes.

Example: Create a new GKE cluster, add a subnet, or update an IAM role.

Tools: GitHub + Jenkins or Cloud Build + Terraform.

Pipeline 2

Application CI/CD pipeline

Triggered when application code changes.

Example: Fix the checkout API and release a new version.

Tools: GitHub + Jenkins or Cloud Build + Docker + ECR/Artifact Registry + EKS/GKE.

These pipelines can also be combined into a single workflow, or infrastructure changes can be handled as a stage within a larger pipeline. The important thing is to run Terraform only when infrastructure changes are required, not automatically recreate infrastructure for every application update.

“In my project, we used Terraform to manage our cloud infrastructure. We maintained the Terraform configuration files in GitHub, and when infrastructure changes were required, we ran a separate pipeline to validate the configuration, review the plan, and apply the changes.

For AWS, Terraform managed resources such as VPCs, IAM roles, EKS clusters, and other required services. For GCP, it managed resources such as VPCs, IAM permissions, GKE clusters, and other cloud services.

Once the infrastructure was ready, our application CI/CD pipeline handled the application code. It built and tested the code, created a Docker image, pushed it to the container registry, and deployed it to Kubernetes.

We did not need to recreate the infrastructure every time a developer changed the application code. We used Terraform whenever the infrastructure needed to be created or updated.”